

Trabajo Práctico 5

- Crea un proyecto Java llamado TP5-ApellidoN (siendo N la inicial de tu nombre).
- Crea un paquete llamado resol.ApellidoN.
- Dentro del paquete resol.ApellidoN, crear una subcarpeta por cada ejercicio.
- Cada carpeta (ejercicioX) debe contener:
 - Una clase EjercicioX.java con un método público y estático llamado ejecutar().
 - Todas las clases auxiliares necesarias para resolver ese ejercicio.
- En la carpeta raíz resol.ApellidoN, crear una clase llamada Principal.java que contenga el método main() y genere un switch para seleccionar qué ejercicio ejecutar.
- Todos los ejercicios deben implementar Programación Orientada a Objetos y utilizar colecciones de Java.
- **Entrega:** Subí este proyecto a Git, en un repositorio que contenga el mismo nombre que el proyecto.

** Nota: deben importar de la siguiente manera las clases para que se ejecuten en la clase principal:

```
//import paquete.carpeta.clase;  
  
import resol.ApellidoN.ejercicio1.Ejercicio1;  
  
import resol.ApellidoN.ejercicio2.Ejercicio2;
```

Ejercicio 1: Tienda de Ropa con Sistema de Compras y Cuotas

Requisitos adicionales:

- El catálogo de prendas debe almacenarse obligatoriamente utilizando una colección de Java.
- Las compras realizadas también deberán almacenarse en una colección para poder listarlas posteriormente.
- El sistema deberá aplicar Programación Orientada a Objetos.
- Debe existir una clase abstracta o interfaz para representar los métodos de pago.
- Los distintos tipos de pago deberán resolverse utilizando herencia y polimorfismo.

Una tienda de ropa en línea desea implementar un sistema que permita a los usuarios:

1. Visualizar un catálogo de ropa utilizando una colección (ArrayList u otra colección apropiada), donde cada prenda tiene:

- Tipo (camisa, pantalón, zapatillas, etc.)
- Precio base
- Talle

2. Proceso de compra:

El usuario podrá seleccionar una prenda del catálogo y elegir el método de pago:

Pago al contado:

Se aplica un 15% de descuento sobre el precio base.

Pago en cuotas:

Se parte del precio base sin descuento y se aplica un recargo acumulativo dependiendo de la cantidad de cuotas elegidas:

- Más de 3 cuotas: recargo del 5%
- Más de 6 cuotas: recargo del 8%
- Más de 12 cuotas: recargo del 12%

3. Cupones de descuento:

Si el usuario aplica un cupón válido, obtiene un 10% de descuento adicional que se aplica:

- Sobre el precio con descuento si paga al contado.
- Sobre el precio base antes de aplicar los recargos por cuotas.

Información a mostrar:

- La prenda elegida (Tipo y talla).
- Precio base.
- Método de pago elegido.
- El descuento aplicado por pago en efectivo (si corresponde).
- La cantidad de cuotas seleccionadas (si el pago es en cuotas).
- El recargo aplicado por el pago en cuotas (si corresponde).
- El descuento aplicado por el uso de un cupón válido (si existe).
- El precio final a pagar después de aplicar todos los descuentos y recargos.

Ejercicio 2: Agenda telefónica de contactos

Se desea implementar un sistema simple de agenda telefónica, con funcionalidades básicas de gestión de contactos. La agenda deberá almacenar los contactos utilizando una colección de Java y deberá estar limitada.

Un contacto está definido por:

- Nombre
- Número de teléfono

La agenda podrá crearse de dos maneras:

- Indicando el tamaño máximo al momento de crearla.
- Con un tamaño por defecto de 10 contactos.

Los métodos de la agenda serán los siguientes:

1. `agregarContacto(Contacto c)`: Añade un contacto a la agenda, sino se pueden meter más a la agenda se indicará por pantalla. No se pueden añadir contactos que dupliquen teléfono.
2. `existeContacto(Contacto c)`: Indica si el contacto pasado existe o no.
3. `listarContactos()`: Lista toda la agenda.

4. `buscaContacto(String nombre)`: Busca un contacto por su nombre y muestra su teléfono.
5. `eliminarContacto(Contacto c)`: Elimina el contacto de la agenda, indica si se ha eliminado o no por pantalla.
6. `agendaLlena()`: Indica si la agenda está llena.
7. `espacioLibre()`: Indica cuántos contactos más podemos agregar.

Método ejecutar:

Implementar un menú interactivo por consola que permita al usuario acceder a todas las funcionalidades mencionadas.